Innovative Evaluation Practices for BCSE308P - Computer Networks Lab

Fall Semester 2026-27

ANALYSIS AND EVALUATION OF CONGESTION-AVOIDANCE MECHANISMS (TAHOE, RENO, CUBIC) WITH A PROPOSED LIGHTWEIGHT PREDICTIVE CONGESTION ESTIMATOR

24BCE0780	**VARSHINI HARIKRISHNA**
24BCE2219	**ANUSKA SEN**
24BDS0066	**ANTRA**

B.Tech.

in

Computer Science and Engineering

School of Computer Science and Engineering



VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

October 2026

ABSTRACT

Classic TCP variants such as Tahoe, Reno and CUBIC treat packet loss as their main congestion signal, so the router queue has usually overflowed by the time they slow down. In this project we asked whether a sender can see congestion coming and back off early. We simulated TCP Tahoe, Reno, CUBIC, Vegas and BBR in NS-3 on a dumbbell topology with a 10 Mbps, 20 ms bottleneck and a 50-packet FIFO queue, under low, medium and high load with ten random seeds each.

From the Tahoe, Reno and CUBIC runs on the first five seeds we built a dataset of 106,354 samples of sender-side RTT measurements taken every 100 ms, each labelled with what the real bottleneck queue did over the next 0.5 s. A decision tree with only four leaves, using two RTT-based features, was trained on seeds 1 to 3 and tested on seeds 4 and 5, and then exported to C++ as a few if/else lines.

We then built Predictive TCP, a Reno-based sender that runs this tree every 100 ms. It cuts its congestion window by 20% when it predicts high congestion, grows slowly when it predicts medium congestion, and keeps Reno's loss response as a safety net. A competition detector switches it to Reno behaviour when a loss-based competitor keeps the queue full.

Compared with Reno in our simulations, Predictive TCP lost about 90% fewer packets and saw 28 to 36% lower delay, while keeping 97 to 100% of Reno's goodput. Its main weakness is fairness: against CUBIC it obtains about 39.6 ± 5.0% of the bandwidth, where 50% would be fair, and the detector can raise false alarms with larger queues. We report these limitations openly.

Keywords: TCP congestion control, NS-3, decision tree, early congestion prediction, fairness.

CERTIFICATE

This is to certify that the project report titled "Analysis and Evaluation of Congestion-Avoidance Mechanisms (Tahoe, Reno, CUBIC) with a Proposed Lightweight Predictive Congestion Estimator" is a bonafide record of the work done by Varshini Harikrishna (24BCE0780), Anuska Sen (24BCE2219) and Antra (24BDS0066) in partial fulfilment of the requirements of the Innovative Evaluation Practices for BCSE308P - Computer Networks Lab during the Fall Semester 2026-2027.

Name and Signature of the Students

TABLE OF CONTENTS

1. INTRODUCTION 6
1.1 Background and motivation 6
1.2 Problem statement 6
1.3 Objectives 6
1.4 Scope and limitations 7
1.5 Mapping to course modules and the lab experiment 7
2. LITERATURE SURVEY 8
2.1 Overview of the existing protocol or scheme 8
2.2 Review of related work 8
2.3 Research gap and justification of novelty 10
3. PROPOSED SYSTEM 11
3.1 System architecture or topology 11
3.2 Baseline mechanism description 11
3.3 Proposed mechanism: algorithm, pseudocode, flowchart 12
3.4 Novelty statement 14
3.5 Assumptions and design decisions 15
4. IMPLEMENTATION 16
4.1 Tools, platform, versions and hardware 16
4.2 Simulation setup 16
4.3 Code structure and explanation of key modules 17
4.3.1 Simulation and variants (C++) 18
4.3.2 Dataset and labels 18
4.3.3 Model training and export 18
4.3.4 Predictive TCP class and settings 19
4.4 Challenges faced and how they were solved 20
5. EXPERIMENTAL SETUP AND PERFORMANCE EVALUATION 21
5.1 Performance metrics and why they were chosen 21
5.2 Scenarios and parameter values 21
5.3 Results 22
5.3.1 All variants side by side 22
5.3.2 Seeds used for choices vs unseen seeds 23
5.3.3 Different queue sizes 23
5.3.4 Sharing the link with CUBIC 24
5.3.5 Ablation at high load 24

5.3.6 Prediction model comparison 25
5.3.7 Graphs to include 25
5.4 Analysis and discussion 28
5.5 Critical evaluation 29
6. CONCLUSION AND FUTURE WORK 31
6.1 Summary of work and key findings 31
6.2 Limitations 31
6.3 Future enhancements 32
7. REFERENCES 33
Appendix A: Important source code and configuration listings 35
Appendix B: Additional screenshots and raw result tables 39

1. INTRODUCTION

1.1 Background and motivation

Data on the internet travels as small packets. When many senders share one link, the router in front of that link keeps waiting packets in a queue. If the queue is full, new packets are dropped and have to be sent again, and a long queue also makes every packet arrive late, which is usually called bufferbloat.

TCP decides how fast each sender transmits, using a congestion window (cwnd) that limits how many packets can be in flight at once. Every TCP variant in this project differs only in how it moves that window up and down. The classic loss-based variants grow the window until a packet is lost and then cut it, which gives the familiar saw-tooth pattern: the queue fills, overflows, drains a little and fills again [1], [2], [11].

The round-trip time (RTT) starts rising as soon as the queue starts filling, well before any loss. At 10 Mbps one 1448-byte packet takes about 1.16 ms to send, so each queued packet adds roughly that much delay and a full 50-packet queue adds about 58 ms. This early signal is what motivated us: if the sender watched its own RTT, it might back off before the queue overflows.

1.2 Problem statement

- Late reaction: Tahoe, Reno and CUBIC need the bottleneck queue to overflow before they slow down.
- Costly side effects: queueing delay, bursts of drops, retransmissions and sudden window collapses (Tahoe, for example, resets to one segment).
- Unused early signals: rising RTT is available at the sender but loss-based TCP does not use it.
- Heavy ML does not fit: deep learning is too slow and too opaque to run inside a sender for every connection, so a predictor has to be small, fast and explainable.

1.3 Objectives

- Simulate and compare TCP Tahoe, Reno and CUBIC under low, medium and high congestion in NS-3.
- Build a dataset of sender-side measurements, labelled with what the real bottleneck queue does 0.5 s later.

- Train a lightweight and explainable model that predicts congestion ahead of time.
- Build Predictive TCP, which uses the prediction to ease off early, and compare it honestly with the existing variants.
- Reduce loss and delay while keeping throughput, and report clearly what happens to fairness.

1.4 Scope and limitations

In scope:

- One NS-3 dumbbell network with a single FIFO bottleneck.
- Tahoe, Reno, CUBIC, Vegas, BBR and Predictive TCP, with three congestion levels and ten random seeds each.
- A Python pipeline for dataset building, model training, C++ export and graphs.

Out of scope:

- Deep learning or reinforcement learning, real hardware, wireless links and multiple bottlenecks.
- Router changes such as AQM or ECN. The mechanism is purely sender-side.

The main limitations (fairness against CUBIC, detector false alarms, a single topology and simulation-only evaluation) are discussed in Sections 5.5 and 6.2.

1.5 Mapping to course modules and the lab experiment

This project belongs to the transport layer part of Computer Networks and extends the lab practice of simulating TCP congestion control in a network simulator. The table below shows where each course topic appears in our work.

Table 1: Course topics covered by the project

Course topic	**Where it appears in the project**
Transport layer: TCP congestion control (slow start, AIMD, fast retransmit and recovery)	Tahoe, Reno and CUBIC baselines, and the Predictive TCP window rules
Queueing and delay at routers	50-packet FIFO bottleneck queue, RTT growth, drops and bufferbloat
Network simulation	NS-3.48 dumbbell topology, custom TCP classes, per-flow tracing
Performance evaluation	Goodput, RTT, drops, retransmissions, Jain fairness index, repeated runs with ten seeds

2. LITERATURE SURVEY

2.1 Overview of the existing protocol or scheme

Jacobson's work introduced the slow start and congestion avoidance behaviour that Tahoe and Reno are built on [1]. Fall and Floyd compared Tahoe, Reno and SACK TCP in simulation [2], which is close in spirit to our baseline study. CUBIC replaced linear window growth with a cubic curve to use fast links better and is standardised in RFC 9438 [5], [13]. Vegas goes the opposite way and reacts to growing delay instead of loss [3], and Mo et al. analysed what happens when Reno and Vegas share a link [4]. Compound TCP mixes a loss-based and a delay-based window [9], and Copa is a more recent delay-based design [8]. BBR estimates bandwidth and minimum RTT and paces at that rate instead of reacting to loss [6].

Table 2: TCP variants studied in this project

Variant	Approach	Main limitation (as we saw it)
Tahoe	Window falls to 1 segment after loss, then slow start	Recovery is slow
Reno	Halves the window on loss, with fast recovery	Reacts only after loss has happened
CUBIC	Cubic window growth for fast links	Can fill shared queues aggressively
Vegas	Uses RTT to estimate queueing delay	Can lose bandwidth to aggressive senders
BBR v1	Estimates bottleneck bandwidth and minimum RTT	Can overflow a shallow queue in our setup

2.2 Review of related work

Learning-based congestion control has grown quickly since 2020. Orca combines a classic controller (CUBIC) with a deep reinforcement learning agent that makes coarse corrections [16], and Sage later tried to learn from the designs of existing heuristic schemes [17]. Recent measurement studies show that coexistence between the BBR family and CUBIC is still being worked on [18], [19]. The table summarises the papers we found most relevant.

Table 3: Literature survey

S.No	Authors, year	Technique	Metrics and tool	Merits	Gaps
1	Jacobson, 1988 [1]	Slow start, congestion avoidance, better RTT and timeout handling	Throughput and stability; real-network experiments	Foundation of Tahoe and Reno; simple and widely adopted	Needs loss to signal congestion
2	Fall and Floyd, 1996 [2]	Simulation comparison of Tahoe, Reno and SACK TCP	Throughput and behaviour with multiple losses; simulator	Shows Reno's weakness with several losses in a window	Loss-based variants only
3	Brakmo and Peterson, 1995 [3]	Vegas: delay-based, compares expected and actual rate	Throughput, loss, queueing delay; experiments	Few losses and small queues	Loses bandwidth to loss-based senders
4	Mo et al., 1999 [4]	Analysis of Reno and Vegas sharing a bottleneck	Bandwidth share and fairness; analysis and simulation	Explains Vegas unfairness against Reno	Older variants only
5	Ha, Rhee and Xu, 2008 [5]	CUBIC: cubic window growth	Throughput, RTT fairness, TCP-friendliness; Linux experiments	Uses fast links well; Linux default	Can fill queues and raise delay
6	Tan et al., 2006 [9]	Compound TCP: loss-based plus delay-based window	Throughput and fairness; experiments	Uses delay inside a hybrid design	Aimed at high-speed, long-distance links
7	Cardwell et al., 2016 [6]	BBR: model bandwidth and minimum RTT, pace at that rate	Throughput and latency; deployment and experiments	High throughput with low delay in deep buffers	Unfair to loss-based flows; can overflow shallow queues [7]
8	Hock, Bless and Zitterbart, 2017 [7]	Experimental evaluation of BBR	Throughput, fairness, queue and loss; testbed	Identifies BBR v1 problems	Evaluation only, no new mechanism
9	Arun and Balakrishnan, 2018 [8]	Copa: delay-based with a target rate	Throughput and delay; experimental evaluation	Low delay with a competitive mode	Fixed design, no learning
10	Abbasloo, Yen and Chao, 2020 [16]	Orca: deep RL corrections on top of CUBIC	Throughput and delay; Internet testbeds and emulation	Adapts across network conditions	Needs a DRL agent; hard to explain

S.No	Authors, year	Technique	Metrics and tool	Merits	Gaps
11	Yen, Abbasloo and Chao, 2023 [17]	Sage: learns from existing heuristic designs	Throughput and delay; experimental evaluation	Learns from classic schemes	Learned model is complex and hard to explain
12	Sarpkaya et al., 2025 [18]	Tests steady-state and fluid models of BBR against CUBIC and Reno, incl. BBRv3	Bandwidth share and fairness; FABRIC testbed	Shows coexistence is still open	Measurement and modelling only
13	BBRv3 study, 2024 [19]	Measurement study of BBRv3	Fairness and convergence; testbed	Reports BBRv3 struggles to coexist with CUBIC	Evaluation only

2.3 Research gap and justification of novelty

- RTT rises as the queue fills, so early warning should be possible.
- Vegas shows that backing off early does not guarantee a fair share.
- BBR v1 shows that high throughput does not guarantee low loss.
- Learning-based schemes such as Orca and Sage use neural networks, which are harder to run inside a sender and harder to explain [15], [16].

We therefore asked whether a tiny, learned and explainable predictor, a four-leaf decision tree that compiles into a few lines of C++, can give low loss and delay without giving up throughput, and what it costs in fairness. We could read the whole model out loud in a viva, and that was a deliberate design choice.

3. PROPOSED SYSTEM

3.1 System architecture or topology

All experiments use a dumbbell topology. Senders connect over 100 Mbps, 1 ms access links to router R1. R1 connects to R2 over a 10 Mbps, 20 ms bottleneck with a 50-packet FIFO queue, and R2 connects to the receivers. The base RTT is about 44 ms and the bandwidth-delay product is about 38 packets of 1448 bytes, so the 50-packet queue is roughly 1.3 times the BDP.

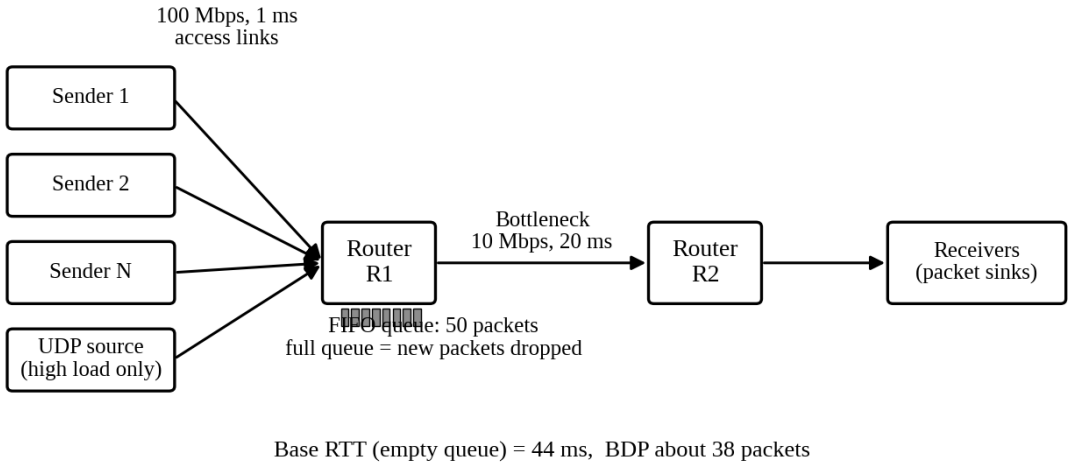


Figure 1: Simulated dumbbell topology

The overall workflow of the project is shown below. Stages 1 to 4 create the model and stages 5 to 8 put it to work and test it.

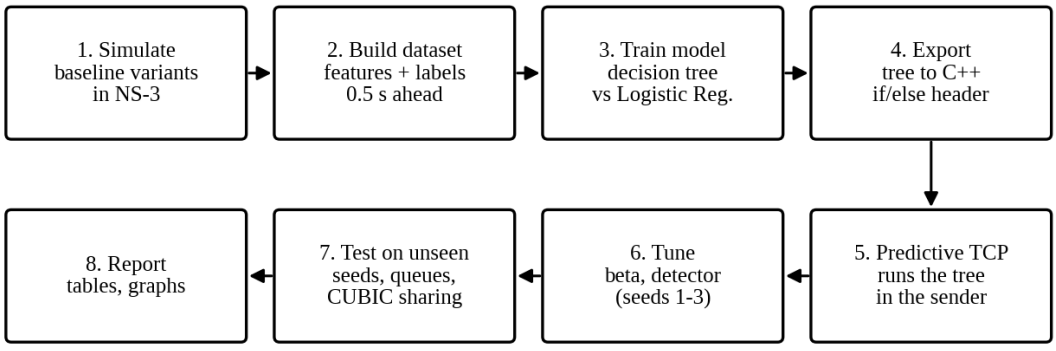


Figure 2: Project pipeline from simulation to report

3.2 Baseline mechanism description

- Tahoe: on three duplicate ACKs the window is set to 1 segment with no fast recovery, so the sender slow-starts again.

- Reno: fast retransmit plus fast recovery. The window is halved on loss and a partial ACK halves it again, which reproduces Reno's known weakness with multiple losses in one window.
- CUBIC: window grows along a cubic curve and is cut to 70% on loss. We used the built-in NS-3 version.
- Vegas and BBR are built-in NS-3 variants that we added as stronger baselines.

NS-3 has neither Tahoe nor classic Reno, so we wrote them as custom recovery classes on top of TcpLinuxReno (Reno's AIMD: plus one segment per RTT, halve on loss). They are close to, but not identical to, the historical implementations.

3.3 Proposed mechanism: algorithm, pseudocode, flowchart

Predictive TCP (class TcpPredictive) extends Reno. Everything Reno does still happens, and a small predictor adds an early and gentle reaction on top. Every 100 ms the sender computes two RTT features from its own measurements and runs a four-leaf decision tree that predicts the congestion level 0.5 s ahead.

Table 4: Features used by the deployed tree

Feature	Meaning
rtt_ratio	Current RTT divided by the lowest RTT seen so far (1.0 means an empty queue)
rtt_vs_avg	Current RTT divided by its own average over the last 1 s (below 1 means the queue is draining)

The tree asks two questions: is rtt_ratio at most 1.23 (then Low); otherwise is rtt_vs_avg at most 0.85 (then Medium, queue draining), and if not, High (queue large and not draining). With a 44 ms base RTT, 1.23 corresponds to about 10 ms of queueing, or roughly 8 packets.

Table 5: Reaction to each prediction

Prediction	What Predictive TCP does	Why
Low	Grows exactly like Reno	No congestion expected, use the link fully
Medium	Grows at 25% of Reno's speed	Queue is draining, but fast growth would refill it. Freezing completely was unfair to other flows
High	Cuts cwnd to 80% (beta = 0.8), at most once per RTT, and stops growing until the prediction changes	Back off before the queue overflows

Safety rules:

- At most one cut per round trip, and never while TCP is already recovering from a loss.
- Never act below a 4-packet window. With tiny windows delayed ACKs inflate the RTT (about 240 ms at one packet) and the model would read that as congestion. We called this the delayed-ACK trap.
- Reno's halving on loss stays as a fallback.

Competition detector: when Predictive TCP shared a link with CUBIC, it saw the queue growing, backed off, and CUBIC took the space it freed. If only Predictive flows share the link, their cuts let the queue drain now and then, so the lowest RTT regularly returns near the base RTT. A loss-based competitor keeps the queue full, so it never does. If even the lowest RTT over the last 2 s stays above 1.4 times the base RTT, the sender assumes a loss-based competitor and behaves exactly like Reno until 10 s after the last such evidence.

Pseudocode:

```
On every ACK:
   record the RTT sample; base_rtt = min(base_rtt, rtt)
   apply Reno window growth, scaled by growth_factor
Every 100 ms:
   rtt_ratio = srtt / base_rtt;  rtt_vs_avg = srtt / average_srtt_over_1s
   if cwnd < MinWindow (4 packets) or in loss recovery: return
   if min RTT over last 2 s > 1.4 * base_rtt: hold_until = now + 10 s
   if now < hold_until: growth_factor = 1 (behave like Reno); return
   class = PredictCongestion(rtt_ratio, rtt_vs_avg)
   if class == Low:    growth_factor = 1
   if class == Medium: growth_factor = 0.25
   if class == High:
      growth_factor = 0
      if one RTT has passed since the last cut: cwnd = max(MinWindow, 0.8 * cwnd)
On packet loss: use Reno's normal response (halve the window)
```

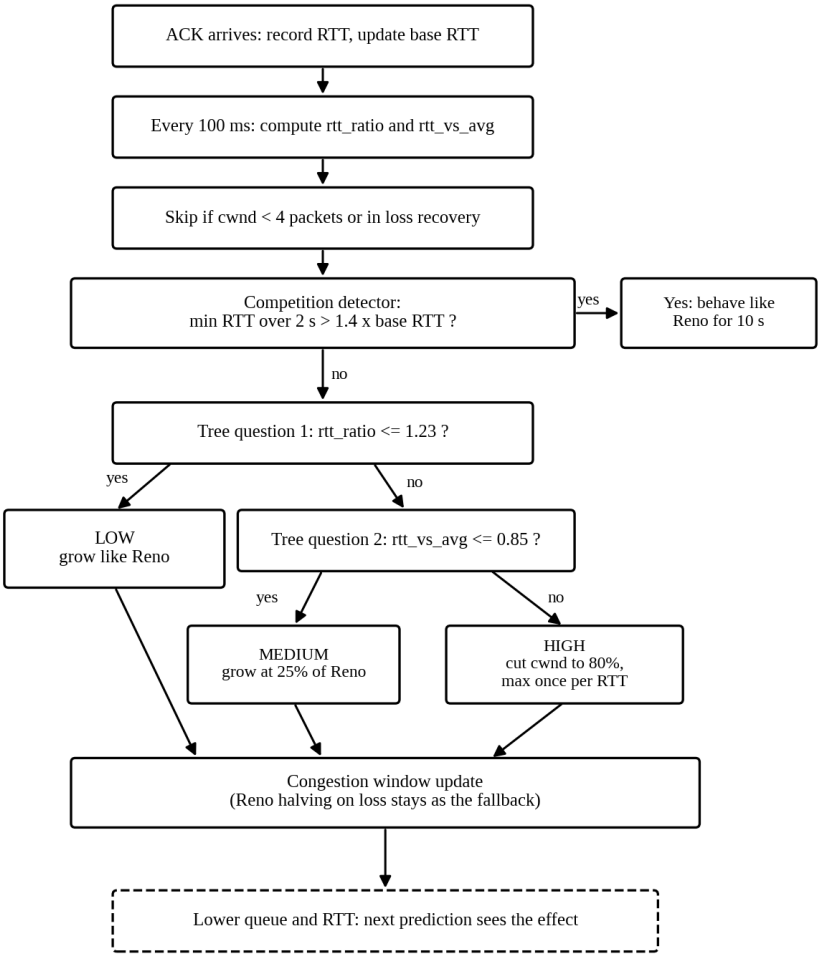


Figure 3: Flowchart of the Predictive TCP control loop

3.4 Novelty statement

The difference from the baseline is that Predictive TCP reacts to a prediction of the queue 0.5 s ahead and not to a loss that has already happened. The prediction comes from a tiny decision tree whose thresholds were learned from simulation data and not hand-picked.

Table 6: Baseline Reno compared with Predictive TCP

Aspect	**Reno (baseline)**	**Predictive TCP (proposed)**
Congestion signal	Packet loss	Predicted queue state from RTT features, loss as fallback
When it reacts	After the queue overflows	About 0.5 s before, on a High prediction
Reaction	Halve window on loss	Cut window by 20% early, grow at 25% on Medium
Model	None	Four-leaf decision tree, no ML library at run time
Competition handling	Not needed	Detector switches to Reno behaviour against loss-based senders

3.5 Assumptions and design decisions

- Only sender-observable inputs are used. The router queue length appears only in the training labels.
- Loss and sender-speed features were left out: loss only describes the past, and the sender's own speed creates a feedback loop (in one early test a flow was throttled to 0.6 Mbps).
- A single decision tree was chosen over a Random Forest or neural network because it exports to a few if/else lines and can be explained.
- A single bottleneck with a FIFO queue is a fair and common model for comparing congestion control.
- The 40% and 80% queue thresholds and the 0.5 s horizon are starting values that we chose, and could be tuned.

4. IMPLEMENTATION

4.1 Tools, platform, versions and hardware

Table 7: Tools and versions

Tool	Version	Use in the project
NS-3 [14]	3.48 (optimized build; modules core, network, internet, point-to-point, applications, traffic-control, flow-monitor)	Simulating routers, links, queues and every TCP sender
C++ (Apple clang 17)	-	Network setup, Tahoe and Reno classes, Predictive TCP
Python	3.13	Dataset building, training, analysis, graphs
scikit-learn, pandas, NumPy, Matplotlib	scikit-learn 1.9.1	Decision tree, Logistic Regression, statistics and plots
Streamlit	-	Optional interactive app that runs live NS-3 simulations
CMake, Ninja, bash scripts	-	build.sh and run_all.sh for one-command reproduction
Hardware	Laptop running macOS	All simulations and training run locally

We used a simulator because it shows the exact queue length at every moment (needed for labels), gives every variant identical traffic, and repeats exactly for a given seed. A 60 s run takes about 1 to 2 s on our laptop.

4.2 Simulation setup

Table 8: Network and TCP parameters

Parameter	Value
Access links	100 Mbps, 1 ms
Bottleneck link	10 Mbps, 20 ms
Bottleneck queue	FIFO, 50 packets (traffic-control queue disc)
Base RTT / BDP	About 44 ms / about 38 packets
Segment size	1448 bytes
Socket buffers	4 MB send and receive
SACK	On for all variants
Minimum RTO	1 s (RFC 6298 [12])
ACKs	Delayed ACKs (NS-3 default)
Run length / seeds	60 s of simulated time / seeds 1 to 10 (ML dataset: seeds 1 to 5)

Table 9: Congestion levels

Level	Traffic	Rough load
Low	3 on/off TCP flows at 2 Mbps each	About 30%
Medium	6 on/off TCP flows at 3 Mbps each	About 90%, bursty
High	8 long-running TCP flows plus 2 Mbps on/off UDP	Saturated
Single	1 TCP flow alone	Checks that one flow can fill the link

The main performance results are aggregated over ten random seeds. For the generalization analysis, seeds 1–3 form the development group and seeds 4–10 form the unseen group. Separately, the ML dataset uses seeds 1–3 for training and seeds 4–5 for testing. Seeds 1 to 3 were used for everything that involved a choice (training, model selection and tuning). Seeds 4 to 10 were never used for any choice and serve only as unseen test seeds. We split the dataset by seed and not by random rows because neighbouring 100 ms rows are nearly identical, and a random split would leak test information into training.

Every 100 ms, for each flow, the simulation writes one CSV row, and at the end of each run a FlowMonitor summary is saved.

Table 10: Data collected

Data	Source	Used for
Smoothed RTT, minimum RTT	TCP socket trace	Features, delay graphs
Congestion window	TCP socket trace	Features, cwnd graphs
Throughput per flow	Packet sink	Throughput graphs
Retransmissions, loss events	TCP socket traces	Loss graphs
Queue length, drops	Bottleneck queue disc	Labels, queue graphs
Lost packets, delay	FlowMonitor	Summary tables

4.3 Code structure and explanation of key modules

Table 11: Project files

File	What it does
ns3/tcp-predictor.cc	Topology, congestion levels, command-line options and 100 ms logging
ns3/tcp-variants.h and .cc	Tahoe and Reno recovery classes, TcpPredictive
ns3/predictive-model.h	Decision tree as C++ code (generated, not edited by hand)
analysis/build_dataset.py	Merges CSVs, computes features and 0.5 s-ahead labels

File	What it does
analysis/train_model.py	Trains the tree and compares it with Logistic Regression and a loss rule
analysis/export_tree.py	Converts the tree to a C++ header and verifies it against scikit-learn
analysis/select_model.py, tune_controller.py, tune_competition.py	Model selection, beta and MinWindow tuning, and detector tuning, all run inside NS-3
analysis/compare_variants.py, robustness.py, make_graphs.py	Metrics and ablation, queue-size and CUBIC tests, graphs and final tables
build.sh, run_all.sh	Compile the simulation and reproduce every result

4.3.1 Simulation and variants (C++)

tcp-predictor.cc builds the dumbbell, installs the on/off, bulk and UDP applications for the chosen level, and takes --variant, --level, --seed, --simTime, --rate, --delay and --queue from the command line. Each run prints a one-line summary and writes CSV files. The same seed always gives the same result.

4.3.2 Dataset and labels

The dataset comes from the Tahoe, Reno and CUBIC baseline runs only (Vegas and BBR are used for comparison, never for training). We ignore the first 3 s of each run and make one sample per flow per 100 ms step, which gave 106,354 samples. Each sample is labelled from the real router queue over the next 0.5 s.

Table 12: Label rules

Label	Rule (next 0.5 s)
High	Queue reaches at least 80% full (40 of 50 packets), or any packet is dropped
Medium	Queue reaches at least 40% full (20 packets) but not High
Low	Neither of the above

Six RTT-based features were computed: rtt_ratio, rtt_grad (RTT change over 100 ms), est_queue_pkts (cwnd x (1 - base RTT / RTT)), rtt_trend_05, rtt_trend_1 and rtt_vs_avg. All are divided by the base RTT where needed so they mean the same on other networks.

4.3.3 Model training and export

- DecisionTreeClassifier with max_depth chosen from 2, 3, 4 and 5 by cross-validation on the training seeds (depth 2 won), min_samples_leaf of 50, class_weight balanced and random_state 0.

- Training used seeds 1 to 3 (64,119 samples) and testing used seeds 4 and 5 (42,235 samples).
- The learned tree uses two of the six features and has four leaves. export_tree.py writes it as C++ and checks that the C++ version gives exactly the same answer as scikit-learn on all 42,235 test samples.
- We also compared the tree with Logistic Regression and a reactive rule that predicts High if a loss happened in the last 0.5 s.

We did not simply pick the best offline scorer. The five best candidate models were each compiled into Predictive TCP and simulated on the training seeds, with the selection rule written down before running: keep candidates with at least 95% of Reno's goodput and fairness of at least 0.9, then pick the largest reduction in drops plus RTT. The Decision Tree warned at least 200 ms before 95% of drop bursts, compared with 38% for the reactive loss-based baseline in Table 22. In separate prediction-horizon experiments, the 0.5 s loss-only model warned at least 200 ms ahead for approximately 67% of drop bursts, while the 0.2 s loss-only model achieved approximately 95%.

4.3.4 Predictive TCP class and settings

TcpPredictive computes the features exactly as the dataset did and calls PredictCongestion() from predictive-model.h. The settings below are NS-3 attributes and can be changed on the command line.

Table 13: Predictive TCP settings

Setting	Default	Meaning
Beta	0.8	Window multiplier on a High prediction
MediumGrowth	0.25	Share of Reno's growth allowed on Medium
MinWindow	4 packets	Predictor never acts below this window
CompetitionWindow	2 s	Detector look-back (0 s turns it off)
CompetitionThreshold	1.4	Lowest RTT divided by base RTT above which a competitor is assumed
CompetitionHold	5 s	Time in Reno mode after the last evidence

4.4 Challenges faced and how they were solved

Table 14: Problems and solutions

Problem	Solution
NS-3 has no Tahoe or classic Reno	Wrote small C++ recovery classes and checked them with cwnd traces
CUBIC misbehaved without SACK (one flow reached only 4.66 Mbps with 10,315 needless retransmissions, against 9.48 Mbps with SACK)	Turned SACK on for all variants
Tahoe froze, because a 200 ms minimum RTO equals the receiver's delayed-ACK timer	Used the standard 1 s minimum RTO
Feedback loop from using the sender's own speed as an input	Model uses only RTT-based network-state features
Delayed-ACK trap at tiny windows	Predictor never acts below 4 packets
Freezing on Medium gave the fewest drops but fairness fell to 0.78	Grow at 25% speed on Medium instead
Best offline model was not the best controller	Chose models by testing them inside NS-3
Beta of 0.5 left a single flow at 87% of Reno's speed	Added a single-flow check and chose beta of 0.8
Only about 23% share against CUBIC	Added the competition detector

Our early plan used SACK off and a 200 ms minimum RTO to match classic behaviour. We changed both after the problems above, and every variant uses the same final settings.

5. EXPERIMENTAL SETUP AND PERFORMANCE EVALUATION

5.1 Performance metrics and why they were chosen

Table 15: Performance metrics

Metric	Meaning	Why we chose it	Better is
Goodput	Useful data delivered per second, without retransmitted copies	Shows whether early backoff costs speed	Higher
Mean RTT	Average round-trip time of packets	The delay users feel; reflects queueing	Lower
Drops	Packets discarded at the bottleneck	Direct measure of queue overflow	Lower
Retransmissions	Packets the senders had to resend	Wasted effort caused by loss	Lower
Mean queue	Average packets waiting at the bottleneck	Shows bufferbloat directly	Lower
Jain fairness index [10]	(sum of rates)^2 divided by (flows x sum of squared rates)	Shows whether flows share the link equally	Closer to 1
Bandwidth share vs CUBIC	Share of total goodput taken by the first variant	Tests competition with loss-based senders	Closer to 50%

For the prediction model we used accuracy, per-class precision, recall and F1, macro-F1, the percentage of drop bursts warned at least 200 ms ahead, and the false alarm rate.

5.2 Scenarios and parameter values

- S1, S2 and S3 are the low, medium and high congestion levels in Table 9. Each variant was run for every level with seeds 1 to 10. The five baseline variants across three levels give 150 baseline runs, and Predictive TCP adds 30 more runs.
- Queue-size test: the model was trained and tuned with a 50-packet queue only, then tested with 20 and 100 packets.
- Competition test: 4 flows of one variant share the link with 4 CUBIC flows at high load. A fair share is 50%.
- Ablation: removing the 20% proactive cut (beta = 1.0) to see what the cut contributes.
- Single-flow test: one flow alone, to check that it can still fill the link.

Predictive TCP uses the default settings in Table 13. Results are reported as the mean and standard deviation over the seeds. Seeds 4 to 10 were never used for any choice.

5.3 Results

Table 16: Results, baseline (Reno) vs proposed (Predictive TCP), mean ± SD over 10 seeds

Scenario	Metric	Baseline (mean ± SD)	Proposed (mean ± SD)	Improvement (%)
S1 (low load)	Goodput (Mbps)	2.91 ± 0.11	2.90 ± 0.11	−0.34%
	Mean RTT (ms)	57.4 ± 1.0	57.6 ± 1.1	−0.35%
	Bottleneck drops	0 ± 0	0 ± 0	N/A*
	Retransmissions	0 ± 0	0 ± 0	N/A*
	Jain fairness index	0.993 ± 0.009	0.993 ± 0.009	0.00%
S2 (medium load)	Goodput (Mbps)	8.49 ± 0.44	8.27 ± 0.28	−2.59%
	Mean RTT (ms)	81.5 ± 4.8	58.7 ± 1.0	27.98%
	Bottleneck drops	660 ± 220	46 ± 44	93.03%
	Retransmissions	663 ± 221	46 ± 45	93.06%
	Jain fairness index	0.986 ± 0.007	0.986 ± 0.008	0.00%
S3 (high load)	Goodput (Mbps)	8.66 ± 0.10	8.62 ± 0.10	−0.46%
	Mean RTT (ms)	101.5 ± 0.6	65.0 ± 2.1	35.96%
	Bottleneck drops	1602 ± 81	164 ± 68	89.76%
	Retransmissions	1413 ± 76	160 ± 67	88.68%
	Jain fairness index	0.995 ± 0.002	0.953 ± 0.023	-4.22%

The results show that Predictive TCP substantially reduces RTT, packet drops, and retransmissions under medium and high loads while maintaining goodput close to Reno. Under low load, both approaches perform similarly because congestion is limited. At high load, Predictive TCP achieves a 35.96% reduction in mean RTT and an 89.76% reduction in bottleneck drops, although fairness decreases slightly.

5.3.1 All variants side by side

Table 17: All six variants at medium and high load

Load	Variant	Goodput (Mbps)	RTT (ms)	Drops	Mean queue (packets)	Fairness
Medium	Tahoe	8.46	81.7	596	25.7	0.985
Medium	Reno	8.49	81.5	660	24.3	0.986
Medium	CUBIC	8.49	85.6	616	26.2	0.985
Medium	Vegas	8.32	64.7	4	6.8	0.987
Medium	BBR	8.39	88.4	2552	27.7	0.985

Load	Variant	Goodput (Mbps)	RTT (ms)	Drops	Mean queue (packets)	Fairness
Medium	Predictive	8.27	58.7	46	6.4	0.986
High	Tahoe	8.56	96.8	1519	40.0	0.990
High	Reno	8.66	101.5	1602	43.3	0.995
High	CUBIC	8.70	109.1	1348	47.0	0.985
High	Vegas	8.67	106.5	255	41.9	0.872
High	BBR	8.56	107.5	7748	47.4	0.848
High	Predictive	8.62	65.0	164	15.2	0.953

The comparison shows that TCP performance varies across congestion-control algorithms and network loads. Predictive TCP achieves the lowest RTT and mean queue length among the listed variants at both medium and high loads. BBR experiences the highest packet drops at both loads, while Predictive TCP maintains relatively high goodput with lower congestion.

5.3.2 Seeds used for choices vs unseen seeds

Table 18: Predictive TCP vs Reno on training and unseen seeds

Load	Seeds	Drops vs Reno	RTT vs Reno	Goodput vs Reno	Fairness
Medium	1–3 (used for choices)	−90.7%	−25.8%	98.1%	0.985
Medium	4–10 (unseen)	−94.3%	−28.8%	97.0%	0.987
High	1–3 (used for choices)	−89.8%	−36.0%	99.7%	0.960
High	4–10 (unseen)	−89.8%	−36.0%	99.5%	0.949

Predictive TCP shows similar improvements on both the seeds used for design choices and the unseen test seeds. It reduces packet drops and RTT while maintaining goodput close to Reno. The consistent results across unseen seeds suggest that the observed benefits are not limited to the initial seed selection.

5.3.3 Different queue sizes

Table 19: Predictive TCP vs Reno with 20, 50 and 100-packet queues

Queue (packets)	Load	Drops vs Reno	RTT vs Reno	Goodput vs Reno
20	Medium	−88.3	−10.5	97.9
20	High	−89.1	−15.8	99.6
50	Medium	−93.0	−27.9	97.4
50	High	−89.8	−36.0	99.6

Queue (packets)	Load	Drops vs Reno	RTT vs Reno	Goodput vs Reno
100	Medium	−97.3	−44.9	97.3
100	High	−40.0	−13.1	100.1

Predictive TCP reduces packet drops and RTT compared with Reno across the tested queue sizes. The strongest reductions occur with a 100-packet queue under medium load. However, the improvements are smaller at high load with the 100-packet queue, indicating that queue capacity and network load influence the effectiveness of the proposed approach.

5.3.4 Sharing the link with CUBIC

Table 20: Bandwidth share against 4 CUBIC flows at high load

Variant (4 flows)	Share of bandwidth taken (%)
CUBIC	48.3 ± 1.9
Reno	44.6 ± 1.0
Vegas	33.6 ± 2.8
BBR	94.5 ± 1.6
Predictive, no detector	23.3 ± 1.4
Predictive, with detector (Phase 8b)	33.5 ± 4.8
Predictive (proposed)	39.6 ± 5.0

The results show that bandwidth sharing depends strongly on the congestion-control variant. BBR takes the largest share of bandwidth, while Predictive TCP with the proposed configuration takes a smaller share and allows the competing CUBIC flows to retain more capacity. The proposed configuration provides a balance between link utilization and competition behavior.

5.3.5 Ablation at high load

Table 21: Effect of the proactive cut

Configuration	Drops	RTT (ms)	Fairness
Predictive TCP (beta = 0.8)	164	65.0	0.953
No proactive cut (beta = 1.0)	276	107.2	0.830

The comparison indicates that the proactive cut improves congestion control under high load. With beta set to 0.8, packet drops decrease from 276 to 164 and RTT decreases from 107.2 ms to 65.0 ms. Fairness also improves from 0.830 to 0.953, showing the benefit of reducing the congestion window proactively.

5.3.6 Prediction model comparison

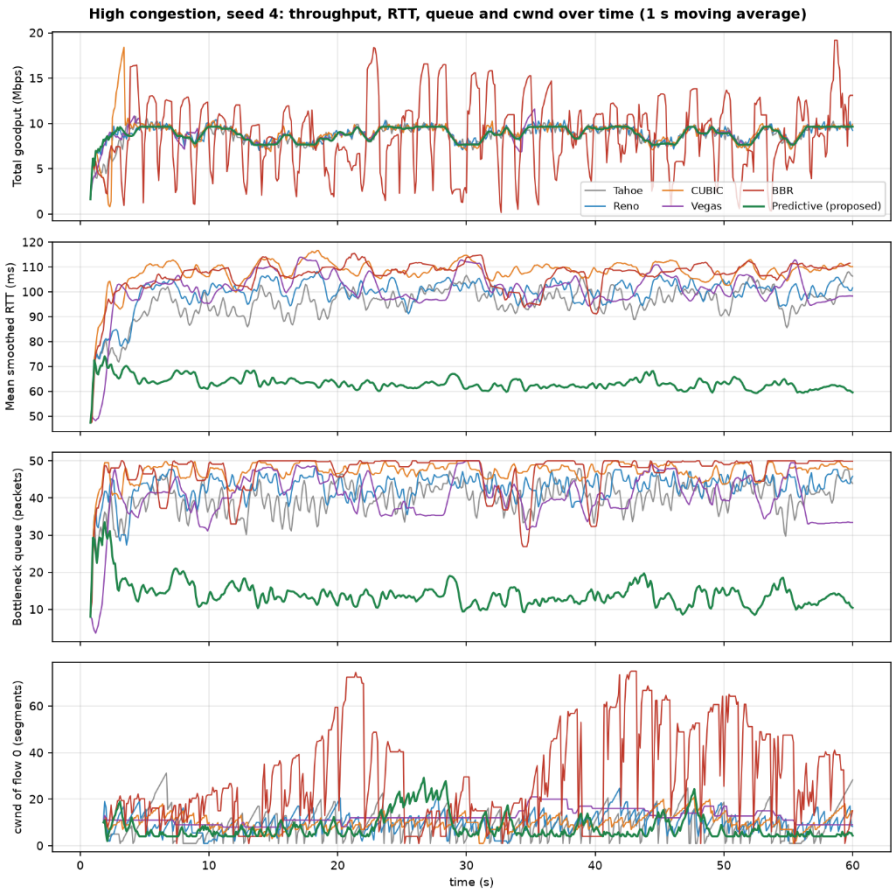
Table 22: Decision tree vs Logistic Regression vs reactive rule

Model	Accuracy	Macro-F1	Precision (High)	Recall (High)	Warned at least 200 ms ahead	False alarms
Decision Tree	0.804	0.605	0.905	0.863	95%	20.5%
Logistic Regression	0.650	0.539	0.930	0.622	85%	13.5%
Reactive loss rule	0.547	0.364	0.915	0.498	38%	11.9%

The Decision Tree achieves the highest accuracy, Macro-F1, recall for the High class, and early-warning rate among the evaluated models. Logistic Regression provides higher High-class precision and fewer false alarms. The reactive loss-based rule performs worst in overall classification and early warning. These results support the Decision Tree as the strongest overall option when early detection is a priority.

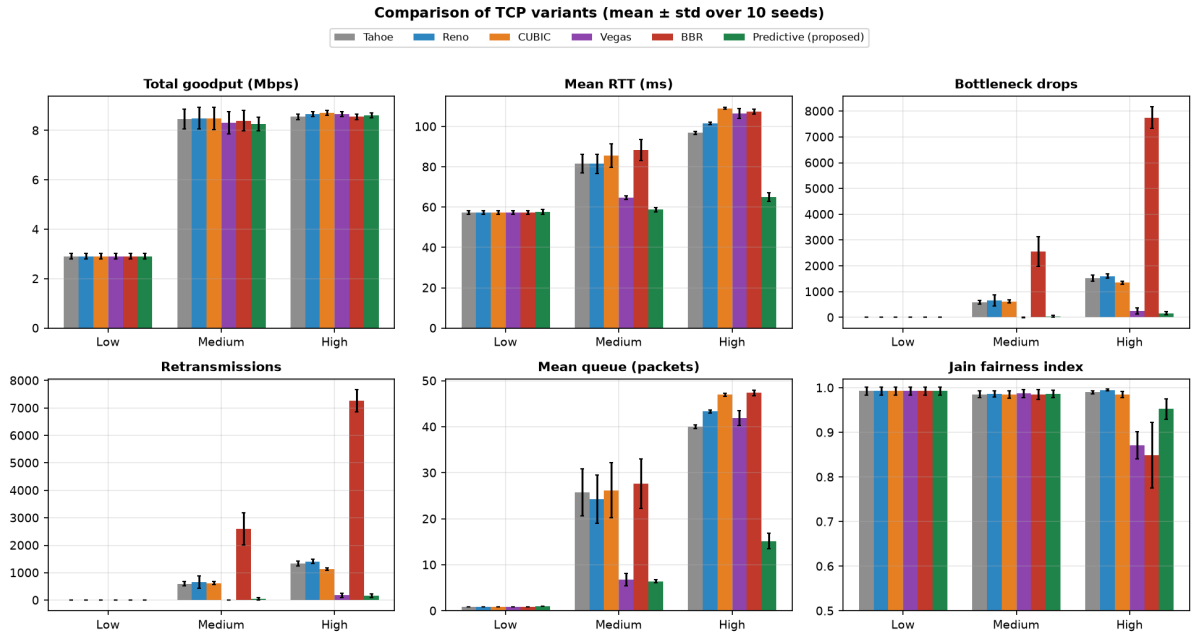
5.3.7 Graphical Results

Figure 4. TCP Performance Time Series Under High Network Load (Seed 4)



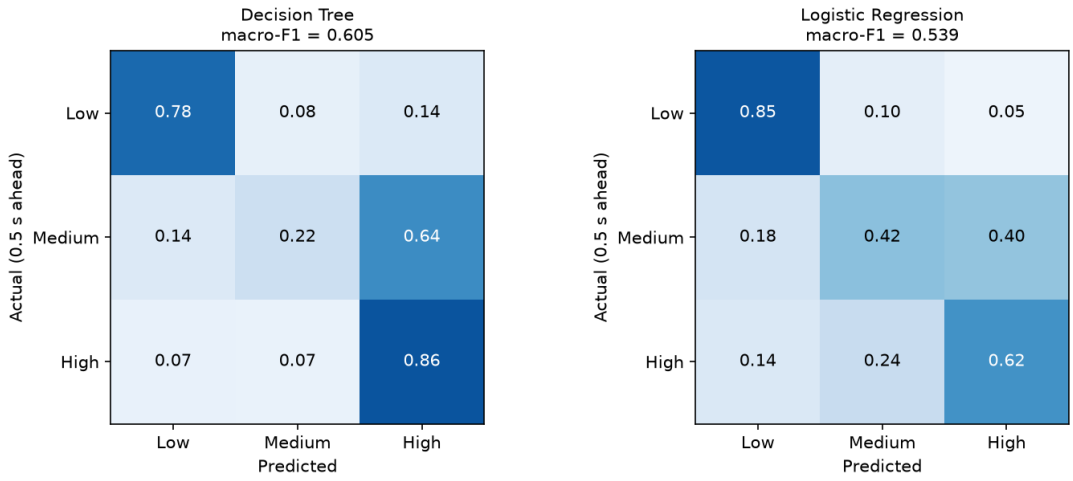
This figure presents the time-series behavior of the TCP variants during a 60-second high-load simulation using seed 4. It illustrates changes in goodput, smoothed RTT, bottleneck queue length, and congestion window (cwnd) over time. These measurements help explain how each congestion-control algorithm responds to network load and congestion.

Figure 5. Performance Comparison of TCP Variants Across Network Loads



This figure compares the mean performance of the evaluated TCP variants across the tested network loads. The bar charts summarize goodput, RTT, packet drops, retransmissions, mean queue length, and fairness, with standard deviations across simulation seeds where shown. The results provide an overall comparison of throughput, delay, congestion, and fairness.

Figure 6. Confusion Matrices for the Prediction Models



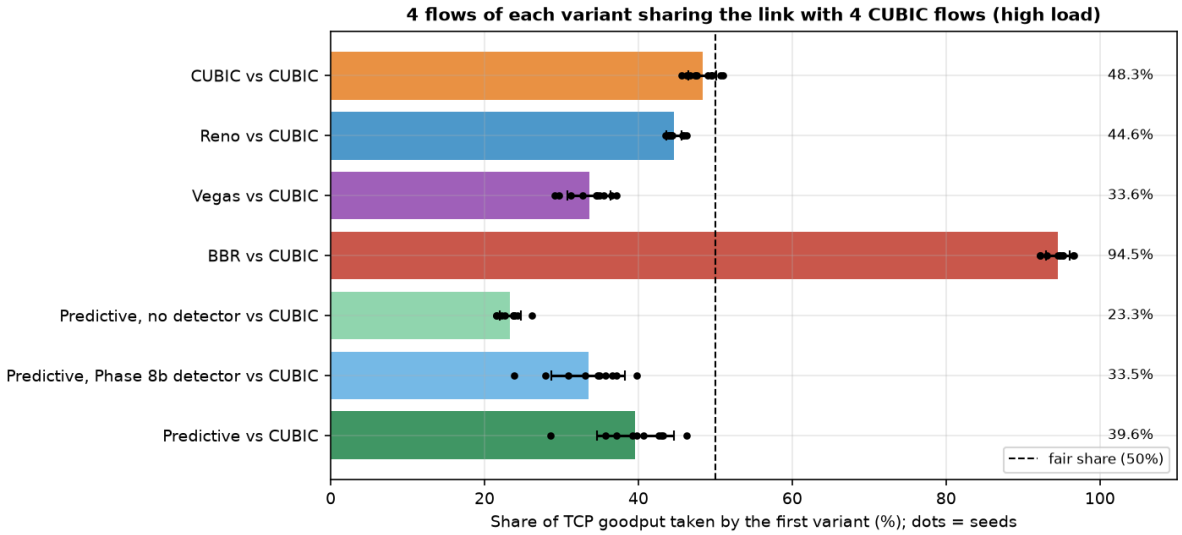
This figure presents the confusion matrices for the Decision Tree and Logistic Regression models evaluated on the test data. The matrices compare predicted classes with actual classes and help identify correct classifications and misclassifications. They provide additional detail for interpreting the classification metrics reported in the model evaluation results.

Figure 7. Deployed Four-Leaf Decision Tree



This figure illustrates the four-leaf decision tree used by the deployed prediction model. The tree shows how network measurements are divided by decision rules to classify network conditions. Its compact structure supports interpretation of the model's decisions and explains how the prediction logic is represented in the implementation.

Figure 8. TCP Bandwidth Sharing Under CUBIC Competition



This figure examines bandwidth sharing when different TCP variants compete with CUBIC flows for bottleneck capacity. The results help assess how fairly each variant shares available

bandwidth and how its behavior affects competing flows. Where individual simulation seeds are plotted, variation between runs can also be examined.

5.4 Analysis and discussion

- Early backoff works. In our simulations Predictive TCP lost about 90% fewer packets than Reno and had 28 to 36% lower RTT, at about 97 to 100% of Reno's goodput. The gains were of the same size on seeds 4 to 10, which were never used for any choice, so we do not think the design is overfitted to the training seeds. See Tables 16 to 18.
- The proactive cut is the main mechanism. Our ablation removes it and the benefit largely disappears, while turning off the special Medium reaction changes very little because the tree rarely predicts Medium.
- Early warning matters more than class accuracy. The delay-only model had the lowest offline macro-F1 but gave the best control, because it warned most drop bursts at least 200 ms ahead.
- The model is close to a delay-based controller. Like Vegas, it acts on queueing delay, but its thresholds and its draining test are learned from data, and its reaction is a 20% cut on top of Reno and not small per-RTT adjustments.
- The tree beat Logistic Regression and the reactive loss rule on accuracy, High recall and early warning, at the cost of more false alarms. Its macro-F1 of about 0.60 is below our 0.70 target, mainly because the Medium class is hard to predict: at medium load congestion often begins when another flow switches on, which one sender cannot foresee. High recall of about 0.86 meets our target.
- BBR v1 showed the opposite failure to Vegas: it keeps the queue full and caused far more drops than Reno in our small-buffer setup, which fits what other studies report [7], [17], [18].
- Vegas was the strongest rival. It lost fewer packets at medium load, while Predictive TCP had lower RTT there and did better at high load. A single Vegas flow also could not fill the link, since delayed ACKs inflate its RTT samples.
- Larger queues give bigger delay gains, because loss-based TCP fills whatever queue exists while Predictive TCP keeps it short.
- Other link speeds and delays keep the drop reduction but expose a fairness problem. In a sweep at high load with 5, 10 and 50 Mbps and 20 and 100 ms delay (queue 1.3 × BDP, 10 seeds), Predictive TCP had fewer drops than Reno in every setting and every

seed (63 to 91% fewer, except 14% at 5 Mbps / 20 ms, where the competition detector false-alarmed 82% of the time), 21 to 33% lower RTT outside that setting, and at least 90% of Reno's goodput. On fast or long links the flows shared unfairly: Jain fairness fell to 0.41 at 50 Mbps / 20 ms (Reno 0.98), 0.37 at 50 Mbps / 100 ms (Reno 0.83) and 0.74 at 10 Mbps / 100 ms (Reno 0.99). The cause is delayed ACKs: a flow with a very small window often waits for the receiver's delayed-ACK timer, so its RTT looks inflated, the tree reads this as congestion and the flow keeps shrinking. A model retrained on a cleaner RTT signal restored fairness on the 50 Mbps link but kept only about half of the main gain, so the deployed model was kept.

- The trade-off is fairness. Against CUBIC, Predictive TCP got about 23% of the bandwidth without the detector and about 40% with it, above Vegas (about 34%) but below Reno (about 45%) and CUBIC (about 48%). Fairness at high load was also slightly lower than Reno's (about 0.95 against 1.00).

5.5 Critical evaluation

Strengths:

- Large reduction in drops, retransmissions and queueing delay with almost the same goodput.
- Tiny, explainable model with no ML library needed at run time.
- Honest testing: For the main performance comparison, seeds 4–10 are treated as the unseen group. This is distinct from the ML model evaluation, where seeds 1–3 are used for training and seeds 4–5 are held out for testing. Choices on seeds 1 to 3 only, results confirmed on seeds 4 to 10, and every run reproducible.

Weaknesses:

- Gives up bandwidth to aggressive loss-based senders such as CUBIC.
- Macro-F1 below target, with poor prediction of the Medium class.
- Tuned on a single topology and traffic mix.

Conditions where it fails or weakens:

- Mixed with CUBIC at high load: share of about 40% instead of 50%. It suits managed networks where most senders behave alike, such as data centres, better than the open internet.

- With a 100-packet queue at high load the competition detector false-alarmed, switching to Reno mode for 82 to 92% of the time in eight of ten seeds, even with no CUBIC flow. The alarm feeds itself: one flow in Reno mode keeps the queue full, so the others think a competitor is present. Predictive TCP still beat Reno in every seed, but by less. The detector can be switched off with CompetitionWindow set to 0 s.
- The number of flows was not varied, and on fast or long links (50 Mbps, or 100 ms delay) the flows share unfairly because delayed ACKs inflate the RTT of small windows (Section 5.4). Beta and MinWindow were tuned only at 10 Mbps / 20 ms and may need other values elsewhere.

6. CONCLUSION AND FUTURE WORK

6.1 Summary of work and key findings

We set out to see whether a tiny machine-learning model could help TCP notice congestion before packets are lost, and in our simulations it could. We built Tahoe and Reno for NS-3, collected a 106,354-sample dataset from the baseline runs, trained a four-leaf decision tree on two RTT features, and built Predictive TCP around it. The key findings are:

- About 90% fewer dropped packets and 28 to 36% lower RTT than Reno, while keeping 97 to 100% of Reno's goodput.
- The gains held on unseen seeds, and for 20, 50 and 100-packet queues.
- The model warned about 95% of drop bursts at least 200 ms ahead, with High recall of about 0.86 and offline macro-F1 of about 0.60.
- Under mixed-flow competition, the final proposed configuration achieves a mean CUBIC bandwidth share of 39.6 ± 5.0%, improving on the earlier Phase 8b detector configuration's 33.5 ± 4.8%, but is still below the fair 50%.

All figures above come from our project results.

Our main takeaway is that predicting congestion early can improve network performance, but a practical design has to balance low delay, throughput and fairness together.

6.2 Limitations

- Simulation only: one NS-3 dumbbell with one FIFO queue. Real networks may differ.
- Fairness against CUBIC is about 39.6 ± 5.0%, improving on the earlier Phase 8b detector configuration's 33.5 ± 4.8%, where 50% is fair, and fairness at high load is slightly below Reno's.
- The competition detector can false-alarm with larger queues.
- Link speed and delay were varied only in one sweep (5 to 50 Mbps, 20 to 100 ms), where fairness fell on fast or long links; the number of flows was not varied.
- Offline macro-F1 is below our target because of the Medium class.
- Tahoe and Reno are our own re-implementations, close to but not identical to the historical versions.

- Labels depend on the 40% and 80% thresholds and the 0.5 s horizon, which we chose as starting values.

6.3 Future enhancements

We have not done the items below yet. They are our planned next steps.

- Test an AQM queue such as FQ-CoDel. Per-flow queueing may let Predictive TCP get closer to a fair share, which would show whether the weakness is specific to plain FIFO queues. This is a hypothesis and not a result.
- Plot bandwidth share against drop reduction for the settings that reach 40 to 44% share, so the trade-off is shown as a frontier and our chosen operating point is a deliberate choice.
- Add a ProbeRTT-style check in Reno mode: briefly shrink the window and see whether the RTT returns near the base. If it does, no competitor is present and the flow can leave Reno mode. This should also reduce false alarms.
- Use graded aggression instead of a binary switch, for example softening beta from 0.8 towards 0.9 the longer a competitor persists.
- Make the detector threshold relative to the largest RTT seen instead of a fixed 1.4 times the base RTT.
- Fix fairness on fast or long links, for example with a per-flow queue-share target instead of the same cut for every flow, vary the number of flows, and report binary congested or not-congested metrics next to macro-F1.
- Validate on a practical testbed and study overhead and stability.

7. REFERENCES

[1] V. Jacobson, "Congestion avoidance and control," in Proc. ACM SIGCOMM, 1988.

[2] K. Fall and S. Floyd, "Simulation-based comparisons of Tahoe, Reno and SACK TCP," ACM Comput. Commun. Rev., 1996.

[3] L. S. Brakmo and L. L. Peterson, "TCP Vegas: End to end congestion avoidance on a global Internet," IEEE J. Sel. Areas Commun., 1995.

[4] J. Mo, R. La, V. Anantharam and J. Walrand, "Analysis and comparison of TCP Reno and Vegas," in Proc. IEEE INFOCOM, 1999.

[5] S. Ha, I. Rhee and L. Xu, "CUBIC: A new TCP-friendly high-speed TCP variant," ACM SIGOPS Oper. Syst. Rev., 2008.

[6] N. Cardwell, Y. Cheng, C. S. Gunn, S. H. Yeganeh and V. Jacobson, "BBR: Congestion-based congestion control," ACM Queue, 2016.

[7] M. Hock, R. Bless and M. Zitterbart, "Experimental evaluation of BBR congestion control," in Proc. IEEE ICNP, 2017.

[8] V. Arun and H. Balakrishnan, "Copa: Practical delay-based congestion control for the Internet," in Proc. USENIX NSDI, 2018.

[9] K. Tan, J. Song, Q. Zhang and M. Sridharan, "A Compound TCP approach for high-speed and long distance networks," in Proc. IEEE INFOCOM, 2006.

[10] R. Jain, D. Chiu and W. Hawe, "A quantitative measure of fairness and discrimination for resource allocation in shared computer systems," DEC Res. Rep. TR-301, 1984.

[11] M. Allman, V. Paxson and E. Blanton, "TCP congestion control," IETF RFC 5681, 2009.

[12] V. Paxson, M. Allman, J. Chu and M. Sargent, "Computing TCP's retransmission timer," IETF RFC 6298, 2011.

[13] L. Xu, S. Ha, I. Rhee, V. Goel and L. Eggert, "CUBIC for fast and long-distance networks," IETF RFC 9438, 2023.

[14] The ns-3 Project, "ns-3 network simulator." [Online]. Available: https://www.nsnam.org/

[15] S. Abbasloo, C.-Y. Yen and H. J. Chao, "Classic meets modern: A pragmatic learning-based congestion control for the Internet," in Proc. ACM SIGCOMM, 2020.

[16] C.-Y. Yen, S. Abbasloo and H. J. Chao, "Computers can learn from the heuristic designs and master Internet congestion control," in Proc. ACM SIGCOMM, 2023.

[17] F. B. Sarpkaya et al., "BBR's sharing behavior with CUBIC and Reno," arXiv:2505.07741, May 2025.

[18] "Promises and potential of BBRv3," in Proc. Passive and Active Measurement Conf. (PAM), 2024. [Online]. Available: https://inet-bbrv3eval.mpi-inf.mpg.de/

Appendix A: Important source code and configuration listings

This appendix lists the most important parts of the implementation. The listings are short excerpts from the project repository; lines marked with "..." were left out for brevity, and a few long lines were wrapped. The default settings are used throughout (Beta 0.8, MinWindow 4, competition detector 2 s / 1.4 / 5 s).

Source code repository: https://github.com/Antra1705/CN. The repository contains the full source code, the scripts that reproduce every result (run_all.sh), and the stored reference data.

A.1 Decision tree exported to C++ (ns3/predictive-model.h, simplified)

The following simplified C++ function is exported from the trained decision tree by analysis/export_tree.py and included in ns3/predictive-model.h. It maps the sender-side RTT features to a congestion class predicted 0.5 seconds ahead.

```
// Returns 0 = Low, 1 = Medium, 2 = High (congestion 0.5 s ahead)
inline int PredictCongestion(double rtt_ratio, double rtt_vs_avg)
{
    if (rtt_ratio <= 1.2333) return 0;        // Low
    else if (rtt_vs_avg <= 0.8548) return 1;  // Medium
    else return 2;                            // High
}
```

A.2 Features and prediction every 100 ms (ns3/tcp-variants.cc, TcpPredictive::Evaluate)

Every 100 ms the sender computes its features from its own RTT measurements only and calls the exported tree. The router queue is never read here; it is used only to label the training data.

```
void TcpPredictive::Evaluate(Ptr<TcpSocketState> tcb)   // called every 100 ms
{
    double sigMs = m_srttMs;      // smoothed RTT now
    double baseMs = m_minRttMs;   // lowest RTT seen = RTT with an empty queue
    double rttRatio = sigMs / baseMs;

    // RTT relative to its own average over the last 1 s (10 samples)
    m_srttHistory.push_back(sigMs);
    if (m_srttHistory.size() > 11)
        m_srttHistory.pop_front();
    size_t n = m_srttHistory.size();
    size_t avgCount = std::min<size_t>(n, 10);
    double rttAvg = std::accumulate(m_srttHistory.end() - avgCount,
                                    m_srttHistory.end(), 0.0) / avgCount;
    double rttVsAvg = sigMs / rttAvg;
    // ... other trained features (RTT gradient, trends, loss counts) ...

    m_prediction = predictive::PredictCongestion(rttRatio, rttGrad, estQueue,
                        cwndSeg, cwndRel, thrMbps, lossRecent, loss2s,
                        rttTrend05, rttTrend1, rttVsAvg, sinceLoss);
    // ... competition detector (A.5) and proactive cut (A.3) follow ...
}
```

A.3 Proactive window cut on a High prediction (TcpPredictive::Evaluate)

When the tree predicts High, the window is multiplied by Beta = 0.8 (a 20% cut), at most once per RTT and never below MinWindow = 4 segments. Reno's normal loss recovery still applies.

```
// React before the queue overflows: shrink the window at most once per RTT,
// and only while TCP is not already handling a loss
bool canReduce = !m_competitive
                 && tcb->m_congState == TcpSocketState::CA_OPEN
                 && now - m_lastReduction >= MilliSeconds(int64_t(m_srttMs));
if (m_prediction == predictive::HIGH && canReduce)
{
    uint32_t reduced = std::max<uint32_t>(uint32_t(cwnd * m_beta),   // Beta = 0.8
                                          m_minWindow * tcb->m_segmentSize);
    if (reduced < cwnd)
    {
        tcb->m_cWnd = reduced;
        tcb->m_ssThresh = reduced;   // leave slow start
        m_lastReduction = now;
    }
}
```

A.4 Window growth for each predicted level (TcpPredictive::IncreaseWindow)

```
void TcpPredictive::IncreaseWindow(Ptr<TcpSocketState> tcb, uint32_t segmentsAcked)
{
    if (m_competitive || tcb->m_cWnd < m_minWindow * tcb->m_segmentSize)
    {
        // Competitor present, or window too small to matter: grow like Reno
        TcpLinuxReno::IncreaseWindow(tcb, segmentsAcked);
        return;
    }
    if (m_prediction == predictive::HIGH)
        return;   // reduced in Evaluate(); no growth until the prediction clears
    if (m_prediction == predictive::MEDIUM && m_holdOnMedium)
    {
        // Grow at 25% of Reno's rate: ACKs are converted to growth credit
        m_ackCredit += segmentsAcked * m_mediumGrowth;
        auto credited = static_cast<uint32_t>(m_ackCredit);
        m_ackCredit -= credited;
        if (credited > 0)
            TcpLinuxReno::IncreaseWindow(tcb, credited);
        return;
    }
    TcpLinuxReno::IncreaseWindow(tcb, segmentsAcked);   // Low: normal Reno growth
}
```

A.5 Competition detector (TcpPredictive::Evaluate)

If the lowest RTT over the last 2 s stays above 1.4 times the base RTT, the queue never drained, so a loss-based sender such as CUBIC is assumed and Predictive TCP behaves like Reno for at least 5 s. The base RTT is the lowest raw RTT sample.

```
// Lowest RTT of each 100 ms interval, kept for the last 2 s (20 intervals)
m_rttFloorHistory.push_back(m_intervalMinRttMs);
while (m_rttFloorHistory.size() > needed)          // needed = 2 s / 100 ms = 20
    m_rttFloorHistory.pop_front();
if (m_rttFloorHistory.size() == needed)
{
    double floorMs = *std::min_element(m_rttFloorHistory.begin(),
                                       m_rttFloorHistory.end());
    double detectorBaseMs = m_rawMinRttMs;         // lowest raw RTT sample
    if (floorMs / detectorBaseMs > m_compThreshold)   // threshold 1.4
        m_lastCompEvidence = now;                  // the queue never drained
    m_competitive = now - m_lastCompEvidence <= m_compHold;   // hold 5 s
}
```

A.6 Dumbbell topology in NS-3 (ns3/tcp-predictor.cc, excerpt)

```
PointToPointHelper access;                         // sender/receiver links
access.SetDeviceAttribute("DataRate", StringValue("100Mbps"));
access.SetChannelAttribute("Delay", StringValue("1ms"));

PointToPointHelper bottleneck;                     // R1 -> R2
bottleneck.SetDeviceAttribute("DataRate", StringValue(bottleneckRate));   // 10Mbps
bottleneck.SetChannelAttribute("Delay", StringValue(bottleneckDelay));    // 20ms
bottleneck.SetQueue("ns3::DropTailQueue", "MaxSize", StringValue("1p"));

for (uint32_t i = 0; i < nSenders; ++i)
    senderLinks.push_back(access.Install(senders.Get(i), routers.Get(0)));
NetDeviceContainer coreLink = bottleneck.Install(routers.Get(0), routers.Get(1));
NetDeviceContainer recvLink = access.Install(routers.Get(1), receiver.Get(0));

// FIFO queue of 50 packets on R1 -> R2: this is where packets wait and are dropped
TrafficControlHelper tch;
tch.SetRootQueueDisc("ns3::FifoQueueDisc", "MaxSize",
                     QueueSizeValue(QueueSize(QueueSizeUnit::PACKETS, queueSize)));
QueueDiscContainer qdiscs = tch.Install(coreLink.Get(0));
```

A.7 Labelling the training data 0.5 s ahead (analysis/build_dataset.py)

Each 100 ms sample is labelled from the real bottleneck queue over the next 0.5 s: Medium if the queue reaches 40% of its capacity, High if it reaches 80% or a packet is dropped.

```
MEDIUM_THRESHOLD = 0.4
HIGH_THRESHOLD = 0.8

def future_labels(df, horizon=HORIZON):            # HORIZON = 0.5 s
    bottleneck = df.drop_duplicates("time").set_index("time").sort_index()
    occupancy = bottleneck.queue_pkts / bottleneck.queue_cap
    steps = int(round(horizon / INTERVAL))         # 5 intervals of 100 ms
    # Max occupancy and total drops over the NEXT `steps` intervals (excluding now)
    future_occ = (occupancy[::-1].rolling(steps, min_periods=steps)
                  .max()[::-1].shift(-1))
    future_drops = (bottleneck.drops[::-1].rolling(steps, min_periods=steps)
                    .sum()[::-1].shift(-1))

    label = pd.Series(0, index=bottleneck.index)                          # Low
    label[future_occ >= MEDIUM_THRESHOLD] = 1                             # Medium
    label[(future_occ >= HIGH_THRESHOLD) | (future_drops > 0)] = 2        # High
```

A.8 Training the decision tree (analysis/train_model.py)

```
TRAIN_SEEDS = [1, 2, 3]
DEPTHS = [2, 3, 4, 5]

def choose_depth(X, y, groups):                    # groups = seed of each sample
    scores = {}
    for depth in DEPTHS:
        tree = DecisionTreeClassifier(max_depth=depth, min_samples_leaf=50,
                                      class_weight="balanced", random_state=0)
        scores[depth] = cross_val_score(tree, X, y, groups=groups,
                                        cv=GroupKFold(n_splits=3),
                                        scoring="f1_macro").mean()
    best = max(scores.values())
    # Smallest depth within 0.005 of the best: lighter model, same quality
    return min(d for d, s in scores.items() if s >= best - 0.005), scores

depth, cv_scores = choose_depth(X_train, y_train, train.seed)   # depth 2 won
tree = DecisionTreeClassifier(max_depth=depth, min_samples_leaf=50,
                              class_weight="balanced", random_state=0)
tree.fit(X_train, y_train)
# analysis/export_tree.py then writes the fitted tree to ns3/predictive-model.h
```

A.9 Running the simulations

Commands used to run single simulations from the NS-3 folder:

```
./ns3 run "tcp-predictor --variant=predictive --level=high --seed=1 --outDir=/tmp"
./ns3 run "tcp-predictor --variant=cubic --level=medium --seed=4 --outDir=/tmp"
./ns3 run "tcp-predictor --variant=predictive --mix=cubic --level=high --outDir=/tmp"
./ns3 run "tcp-predictor --variant=predictive --level=high --queue=100 --outDir=/tmp"
```

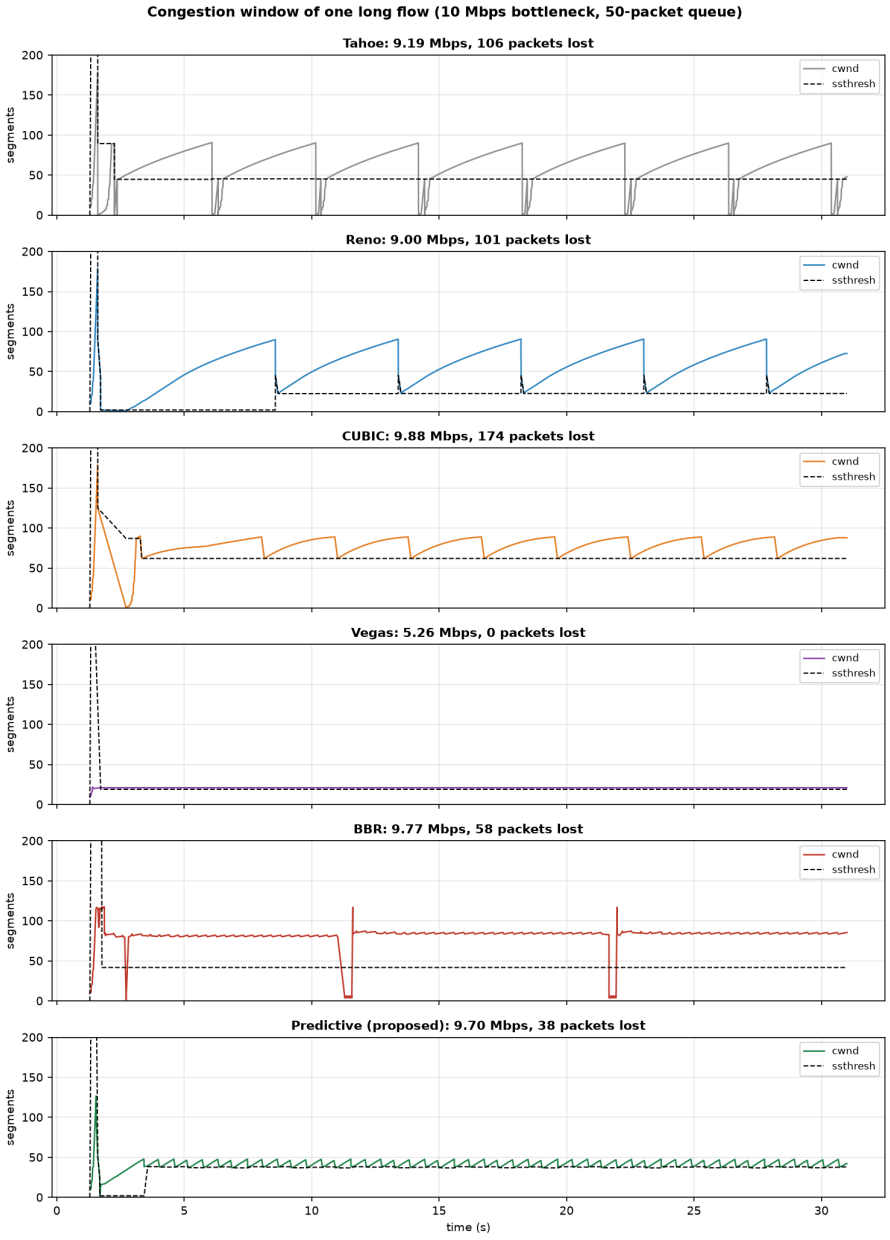
To rebuild and reproduce every result: ./build.sh followed by ./run_all.sh. The option ./run_all.sh --reference reproduces the original results from the stored reference data without NS-3.

Appendix B: Additional screenshots and raw result tables

This appendix collects screenshots of the simulator and the interactive app, together with the raw per-seed results behind the averages reported in Chapter 5. All values come from the project's results folder (10 seeds, 60 s per run, 10 Mbps / 20 ms bottleneck with a 50-packet queue).

B.1 Validation of the TCP implementations

Figure 9. Congestion Window of a Single Long Flow for Each TCP Variant

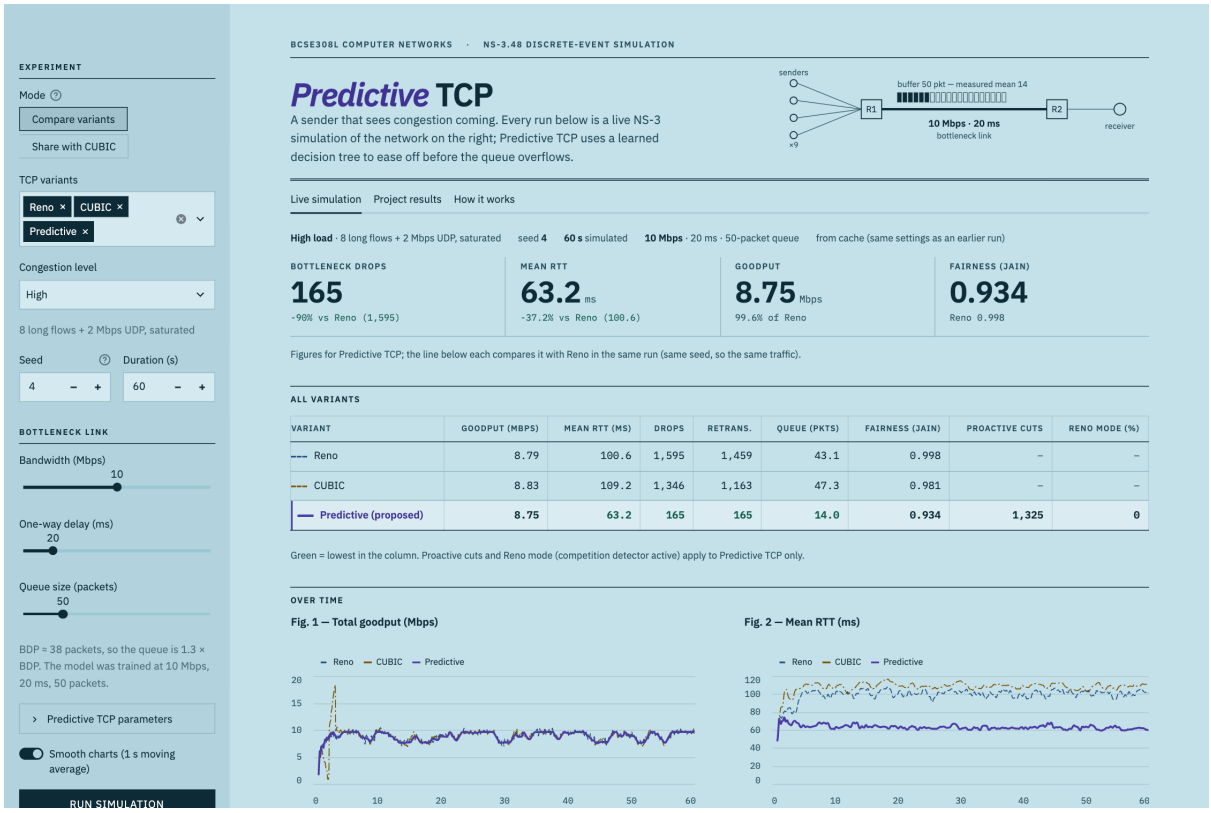


This figure validates the six implementations with one flow alone on the bottleneck. Tahoe falls back to slow start after every loss, Reno halves its window and gives the classic saw-tooth,

CUBIC grows along its cubic curve, Vegas holds a steady window with no losses, and BBR keeps its window near the estimated bandwidth-delay product with periodic ProbeRTT dips. Predictive TCP keeps a smaller window with small early cuts: it lost 38 packets against 101 for Reno in the same run, while reaching 9.70 Mbps against 9.00 Mbps.

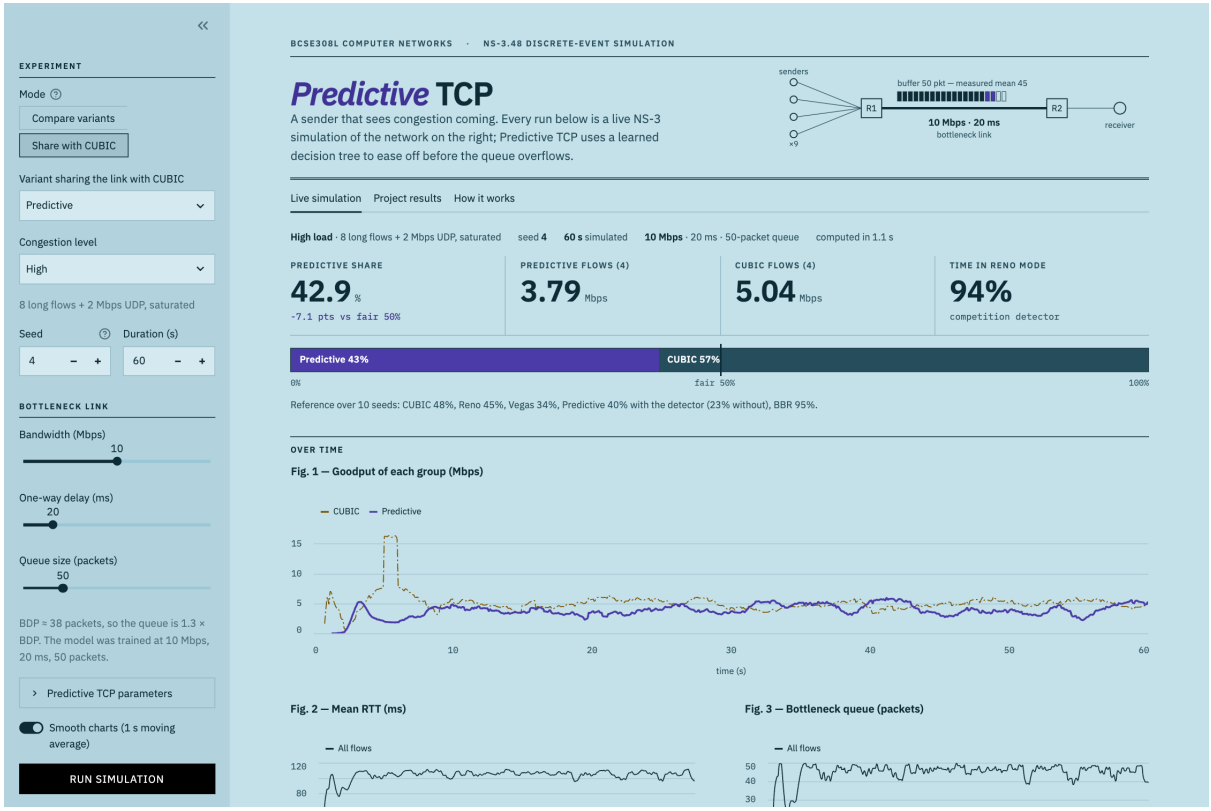
B.2 Screenshots of the interactive simulator

Figure 10. Live NS-3 Run in the Streamlit App (High Load, Seed 4)



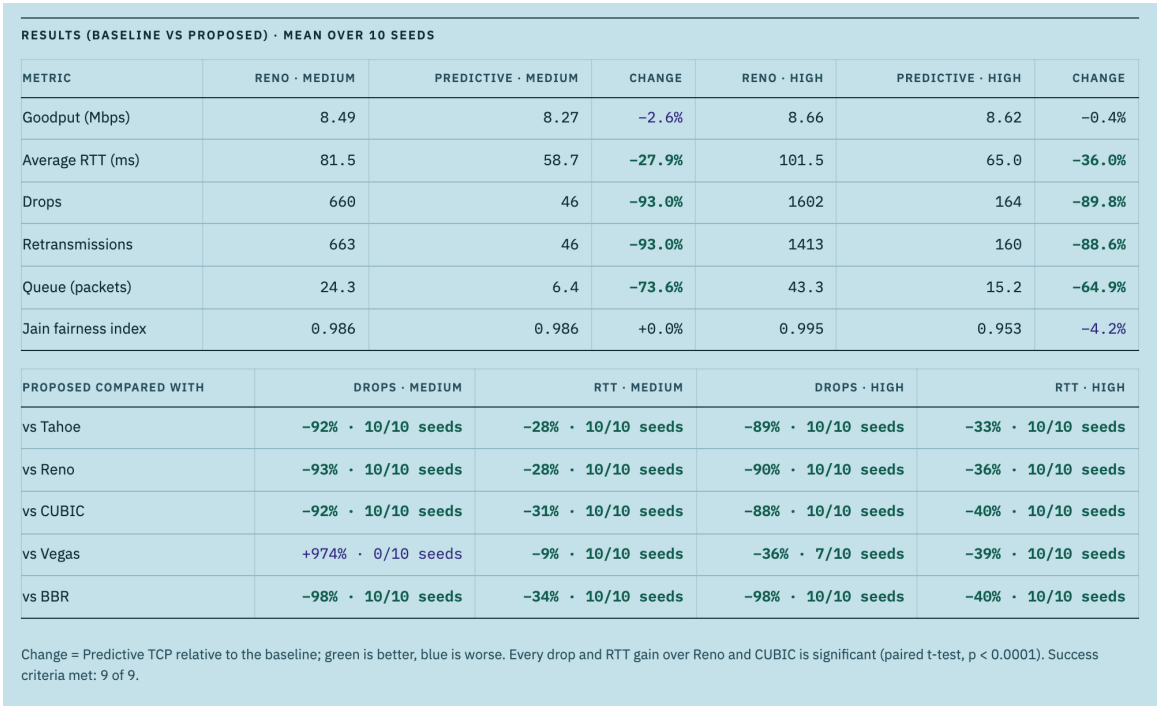
This screenshot shows a live run of the interactive simulator for Reno, CUBIC and Predictive TCP at high load on seed 4, a seed never used for training or tuning. In this run Predictive TCP had 165 bottleneck drops against 1,595 for Reno (-90%) and a mean RTT of 63.2 ms against 100.6 ms (-37.2%), while keeping 99.6% of Reno's goodput. The network diagram at the top shows the measured mean queue of about 14 packets.

Figure 11. Sharing the Link with CUBIC in the App (Competition Detector On)



This screenshot shows four Predictive TCP flows sharing the bottleneck with four CUBIC flows at high load (seed 4). With the competition detector on, Predictive TCP received 42.9% of the goodput and spent 94% of the time in Reno mode; with the detector switched off it received only 21.5% in the same scenario. A fair share would be 50%.

Figure 12. Project Results Tab: Baseline vs Proposed Summary



RESULTS (BASELINE VS PROPOSED) · MEAN OVER 10 SEEDS

METRIC	RENO · MEDIUM	PREDICTIVE · MEDIUM	CHANGE	RENO · HIGH	PREDICTIVE · HIGH	CHANGE
Goodput (Mbps)	8.49	8.27	−2.6%	8.66	8.62	−0.4%
Average RTT (ms)	81.5	58.7	**−27.9%**	101.5	65.0	**−36.0%**
Drops	660	46	**−93.0%**	1602	164	**−89.8%**
Retransmissions	663	46	**−93.0%**	1413	160	**−88.6%**
Queue (packets)	24.3	6.4	**−73.6%**	43.3	15.2	**−64.9%**
Jain fairness index	0.986	0.986	+0.0%	0.995	0.953	−4.2%

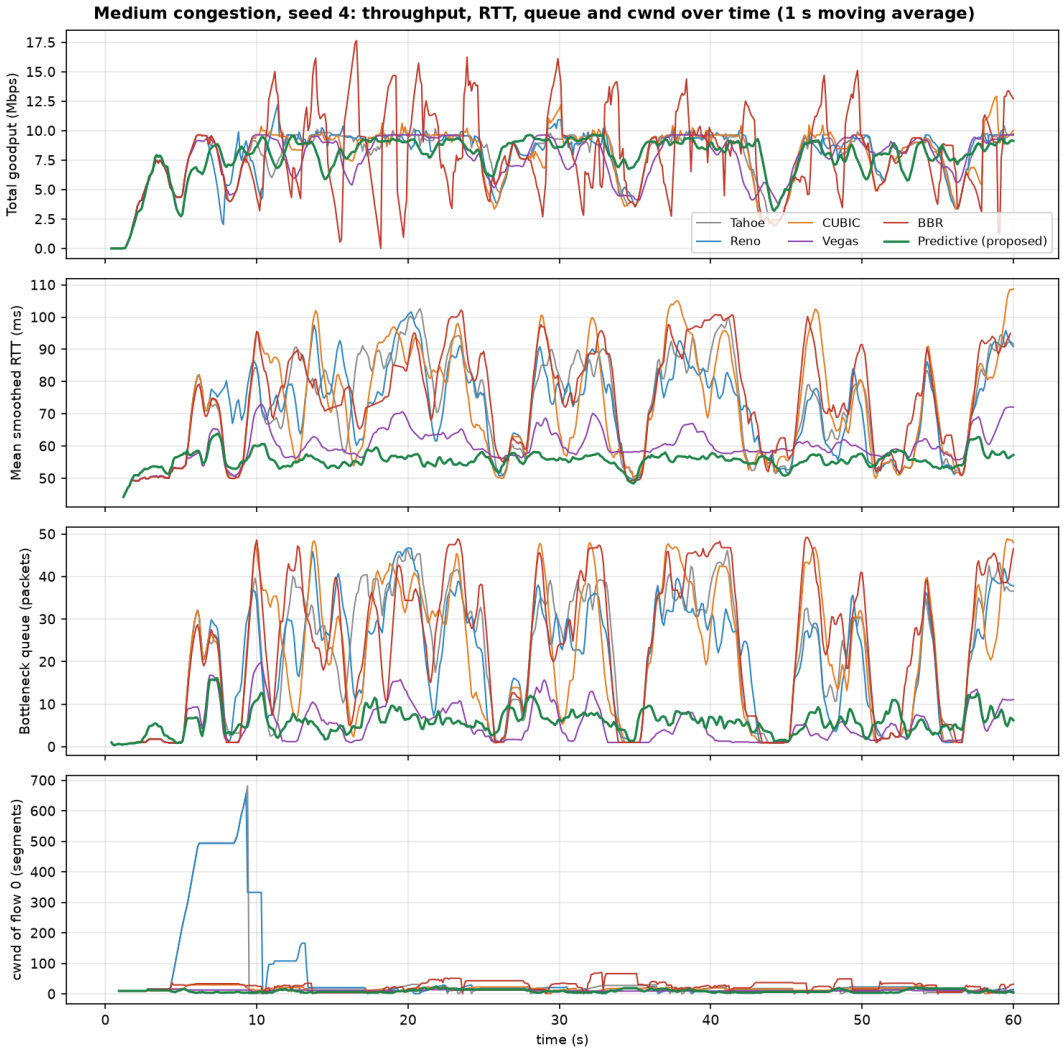
PROPOSED COMPARED WITH	DROPS · MEDIUM	RTT · MEDIUM	DROPS · HIGH	RTT · HIGH
vs Tahoe	**−92% · 10/10 seeds**	**−28% · 10/10 seeds**	**−89% · 10/10 seeds**	**−33% · 10/10 seeds**
vs Reno	**−93% · 10/10 seeds**	**−28% · 10/10 seeds**	**−90% · 10/10 seeds**	**−36% · 10/10 seeds**
vs CUBIC	**−92% · 10/10 seeds**	**−31% · 10/10 seeds**	**−88% · 10/10 seeds**	**−40% · 10/10 seeds**
vs Vegas	+974% · 0/10 seeds	**−9% · 10/10 seeds**	**−36% · 7/10 seeds**	**−39% · 10/10 seeds**
vs BBR	**−98% · 10/10 seeds**	**−34% · 10/10 seeds**	**−98% · 10/10 seeds**	**−40% · 10/10 seeds**

Change = Predictive TCP relative to the baseline; green is better, blue is worse. Every drop and RTT gain over Reno and CUBIC is significant (paired t-test, $p < 0.0001$). Success criteria met: 9 of 9.

This screenshot shows the summary panel of the app's Project results tab, computed from the final 10-seed results. The upper table compares Reno with Predictive TCP for every metric at medium and high load; the lower table shows the change in drops and RTT against each baseline, with the number of seeds (out of 10) in which Predictive TCP was better.

B.3 Queue and RTT traces

Figure 13. Time Series Under Medium Network Load (Seed 4)



This figure shows goodput, mean RTT, bottleneck queue and the congestion window of one flow over a 60-second medium-load run on seed 4, for all six variants. The bursty on/off traffic repeatedly fills the queue to 40-50 packets for the loss-based variants, while Predictive TCP keeps it mostly below 10 packets and its RTT close to the 44 ms base RTT, with almost the same goodput.

B.4 Raw per-seed results

Tables 23 and 24 list the per-seed results of Reno and Predictive TCP that are averaged in Table 16. Seeds 1-3 were used for training and tuning; seeds 4-10 were never used for any design choice. Pred. = Predictive TCP.

Table 23: Per-seed results at medium load, Reno vs Predictive TCP

Seed	Use	Drops Reno	Drops Pred.	RTT Reno (ms)	RTT Pred. (ms)	Goodput Reno (Mbps)	Goodput Pred. (Mbps)	Jain Pred.
1	training	1236	143	80.3	58.8	8.42	8.12	0.973
2	training	574	24	82.3	58.0	8.74	8.57	0.988
3	training	525	50	76.7	60.7	7.97	7.97	0.993
4	unseen	604	0	77.6	58.9	7.92	7.85	0.973
5	unseen	567	89	82.7	59.7	8.55	8.27	0.981
6	unseen	792	65	91.4	57.7	9.28	8.56	0.987
7	unseen	670	14	85.2	57.8	8.93	8.55	0.998
8	unseen	530	50	79.0	59.5	8.48	8.30	0.987
9	unseen	484	1	84.6	57.8	8.63	8.51	0.991
10	unseen	620	26	75.2	58.5	8.02	7.99	0.990
Mean	**all**	**660.2**	**46.2**	**81.5**	**58.7**	**8.49**	**8.27**	**0.986**

Table 24: Per-seed results at high load, Reno vs Predictive TCP

Seed	Use	Drops Reno	Drops Pred.	RTT Reno (ms)	RTT Pred. (ms)	Goodput Reno (Mbps)	Goodput Pred. (Mbps)	Jain Pred.
1	training	1718	200	101.0	66.5	8.61	8.60	0.938
2	training	1685	143	101.6	63.8	8.58	8.53	0.961
3	training	1595	168	102.3	64.9	8.49	8.46	0.982
4	unseen	1595	165	100.6	63.2	8.79	8.75	0.934
5	unseen	1520	139	101.6	63.7	8.81	8.76	0.960
6	unseen	1487	126	100.7	64.4	8.68	8.65	0.978
7	unseen	1488	138	101.8	62.4	8.64	8.57	0.929
8	unseen	1639	34	101.3	68.2	8.63	8.62	0.934
9	unseen	1633	235	102.2	64.1	8.77	8.72	0.928
10	unseen	1655	287	101.5	68.6	8.60	8.55	0.982
Mean	**all**	**1601.5**	**163.5**	**101.5**	**65.0**	**8.66**	**8.62**	**0.953**

The complete raw results for all six variants are in results/comparison_runs.csv in the project repository, with one row per variant, load level and seed.